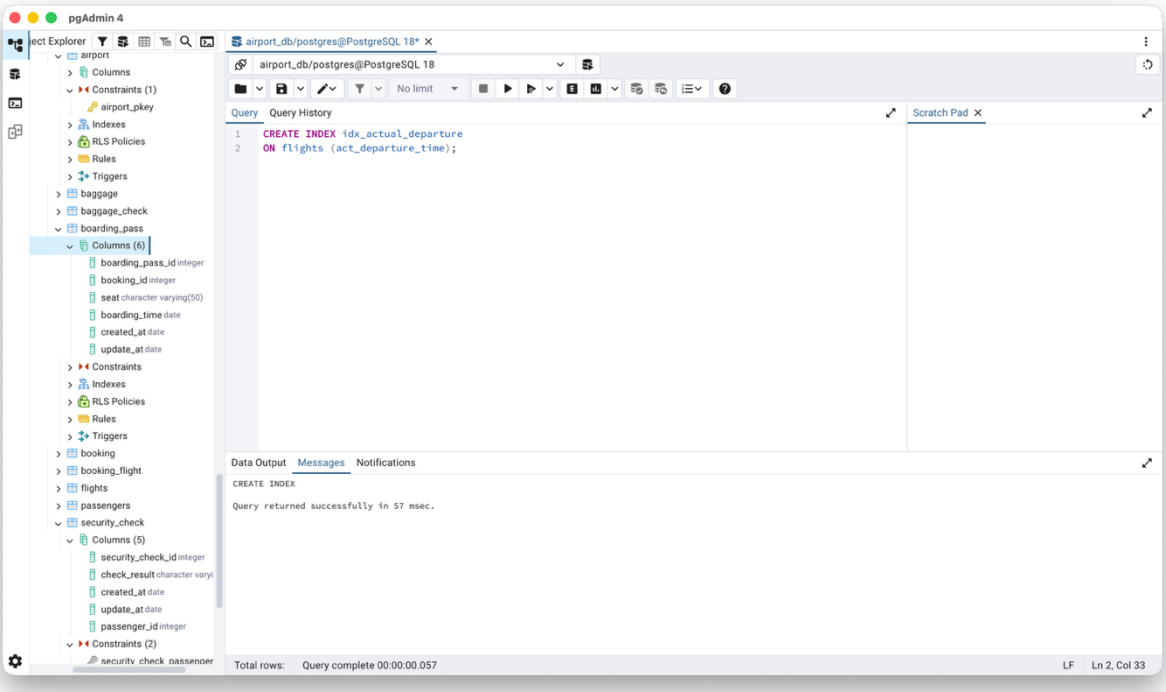
Laboratory work 7

Tasks:

INDEX

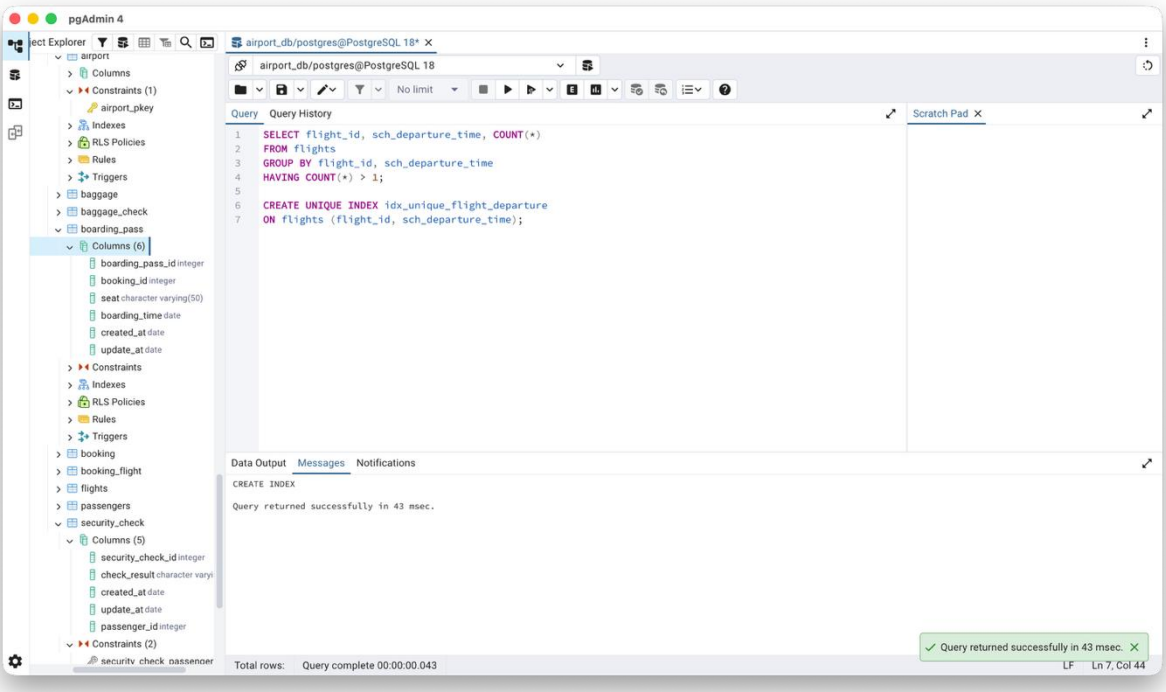
1. Create an index on the actual_departure column in the flights table.

Step 1 - Create an index for actual departure time



2. Create a unique index to ensure flight_no and scheduled_departure combinations are unique.

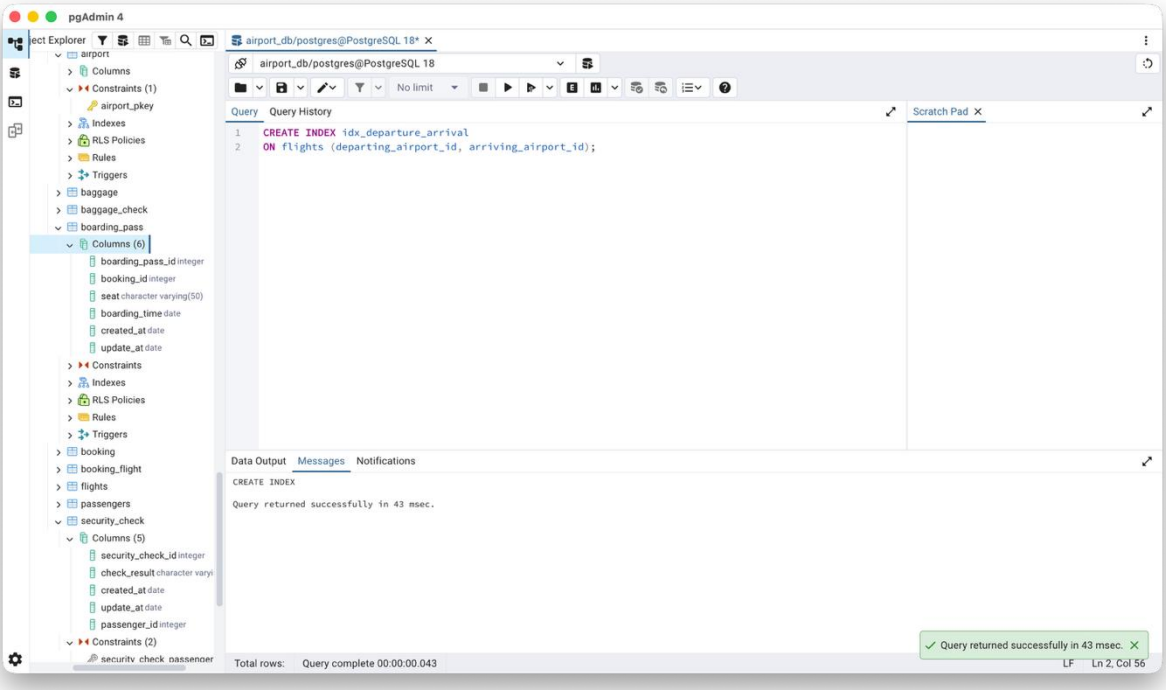
Step 1 - Check and create a unique index



Explanation: This unique index uses flight_id and the scheduled departure time because flight_no is not in my flights table. It is an alternative, not the exact index in the task.

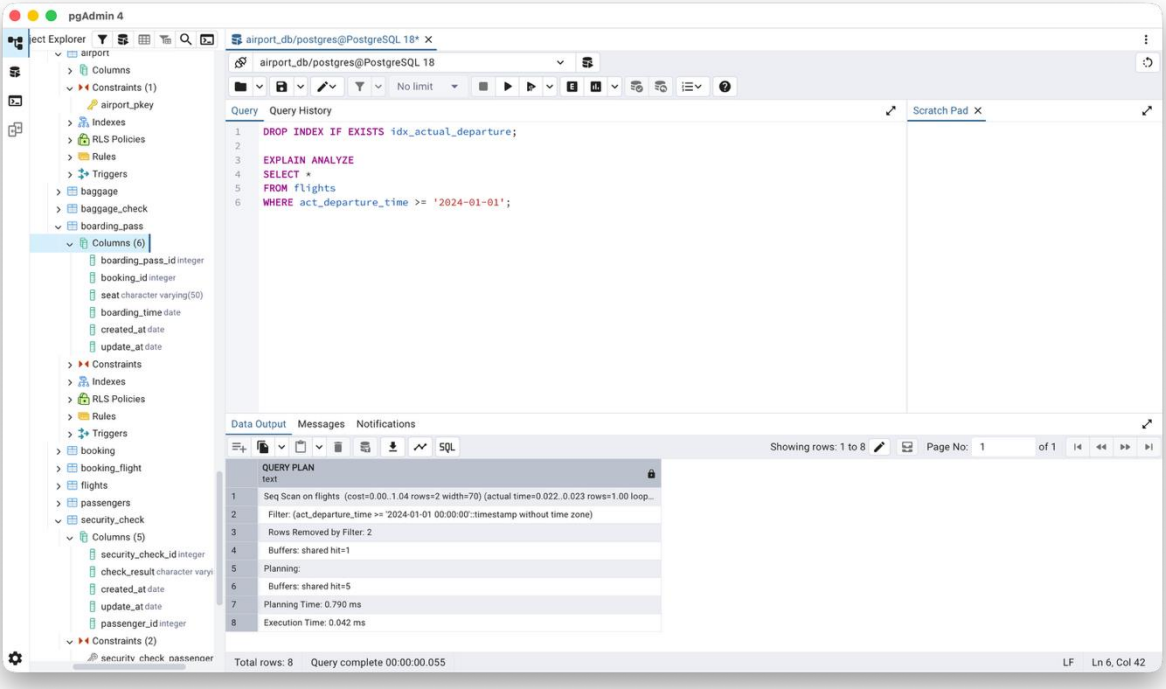
3. Create a composite index on the departure_airport_id and arrival_airport_id columns.

Step 1 - Create an index for departure and arrival airports

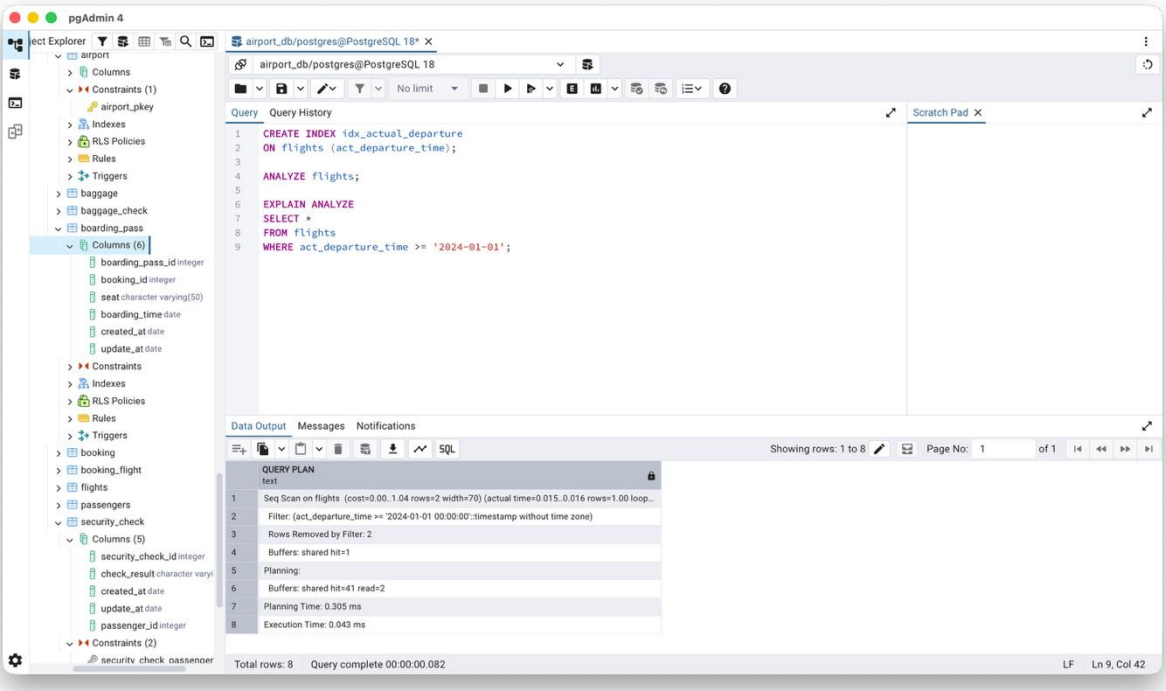


4. Evaluate the difference in query performance with and without indexes. Measure performance differences.

Step 1 - Test the query without the index



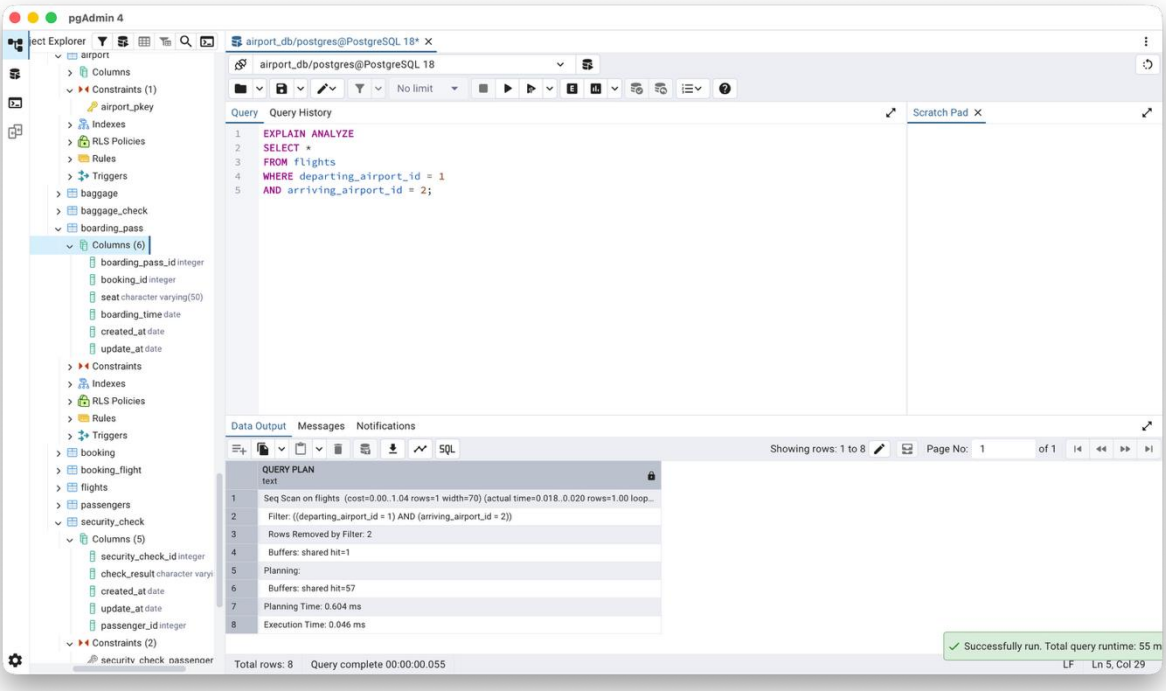
Step 2 - Test the query after creating the index



Explanation: I compared the execution plans before and after creating the index. PostgreSQL can still use a Sequential Scan when the table is small. The Execution Time values show the measured times.

5. Use EXPLAIN ANALYZE to check index usage in a query filtering by departure_airport and arrival_airport.

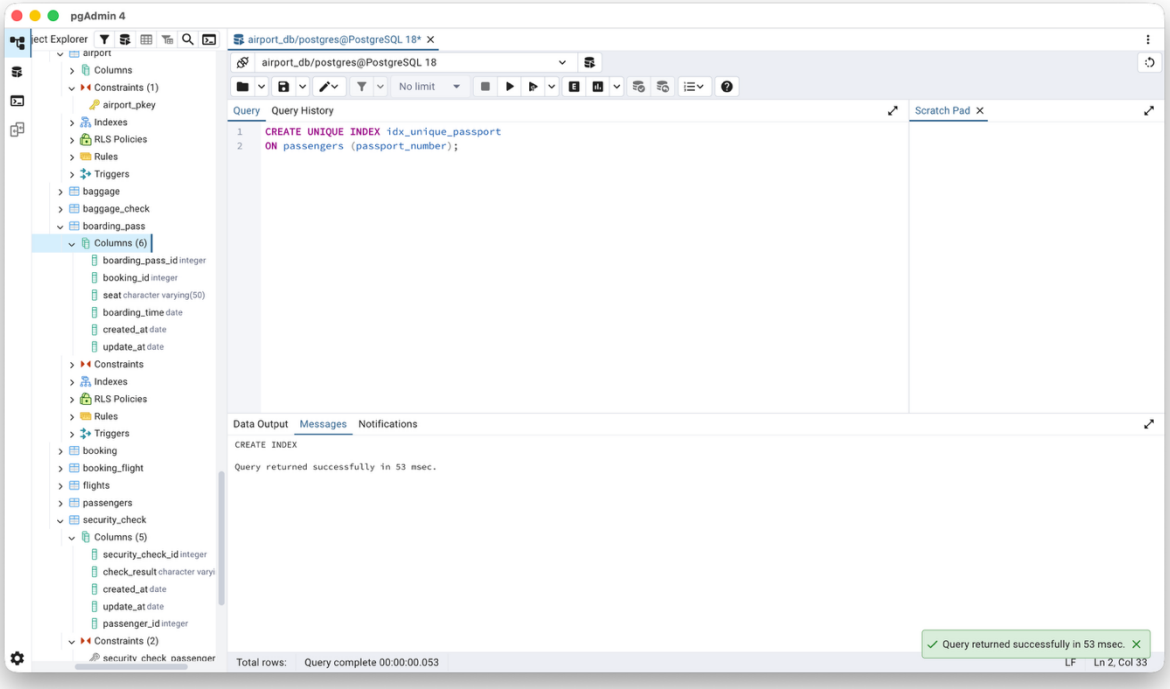
Step 1 - Check the query plan with EXPLAIN ANALYZE



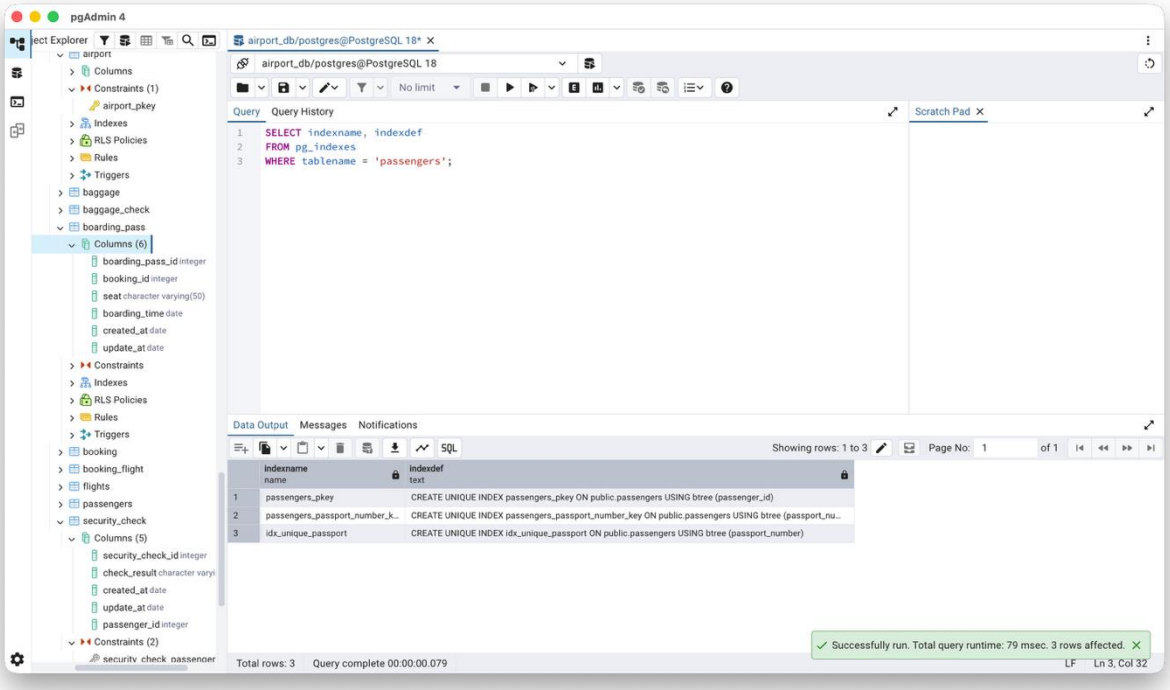
Explanation: EXPLAIN ANALYZE shows how PostgreSQL reads the flights table. The result shows a Sequential Scan, so this query did not use the new index.

6. Create a unique index for the passport_number of the Passengers table. Check if the index was created or not. Insert into the table two new passengers.
   Explain in your own words what is going on in the output?

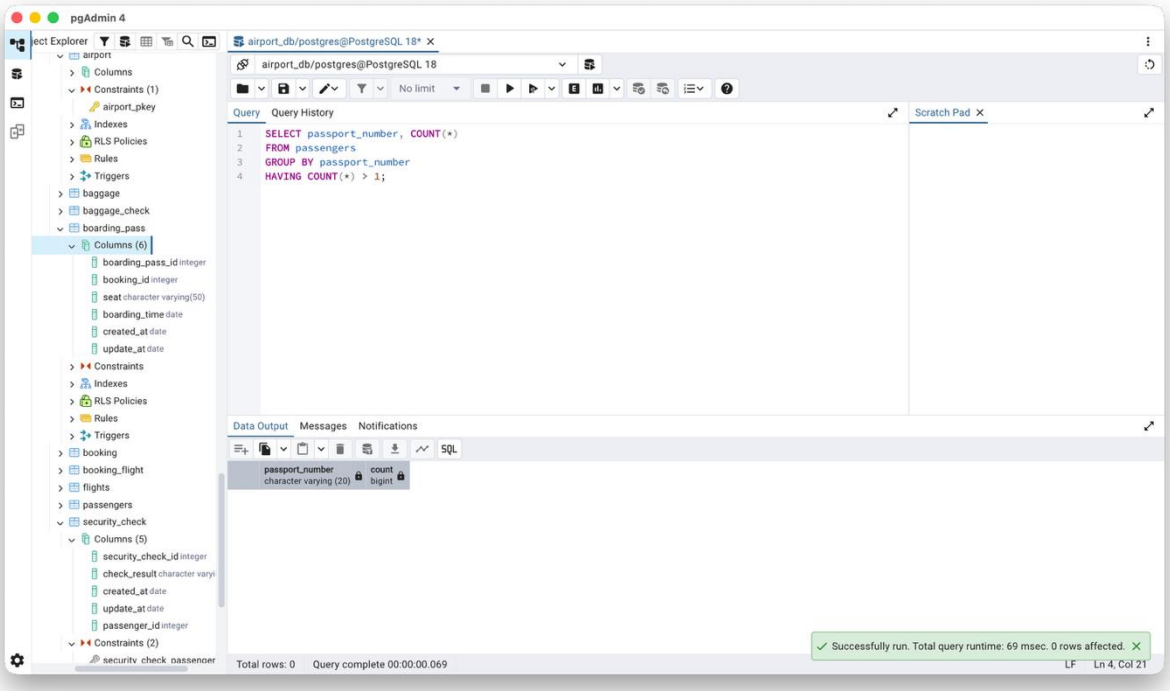
Step 1 - Create a unique passport index



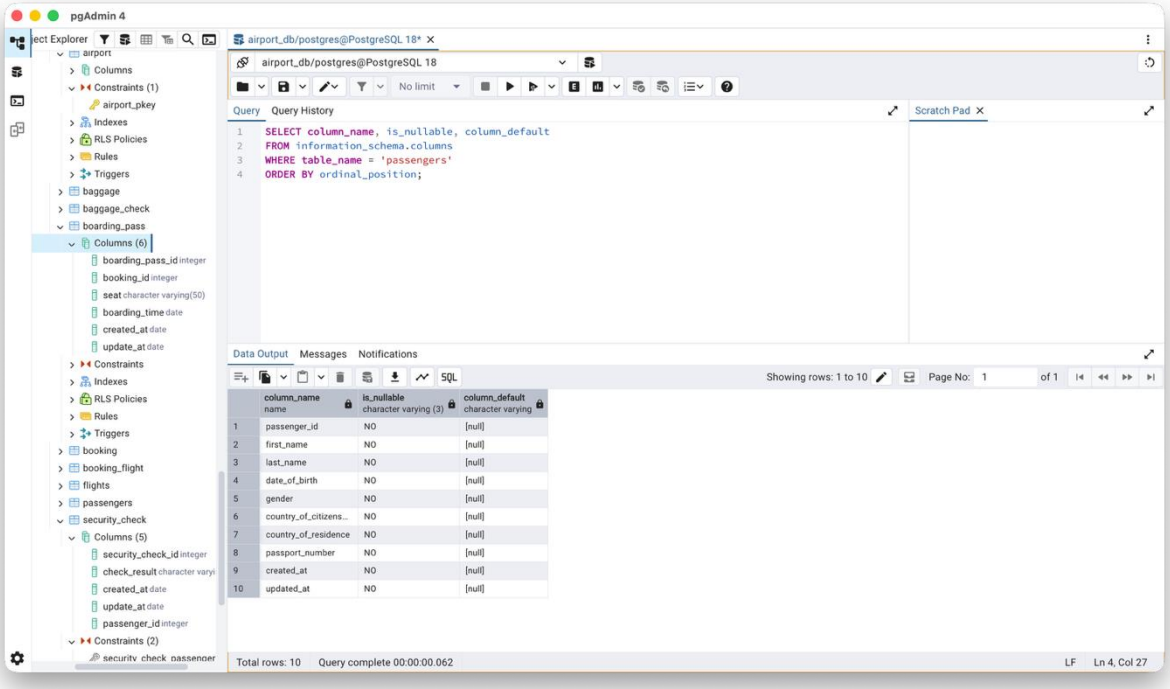
Step 2 - Check indexes in the passengers table



Step 3 - Review the result



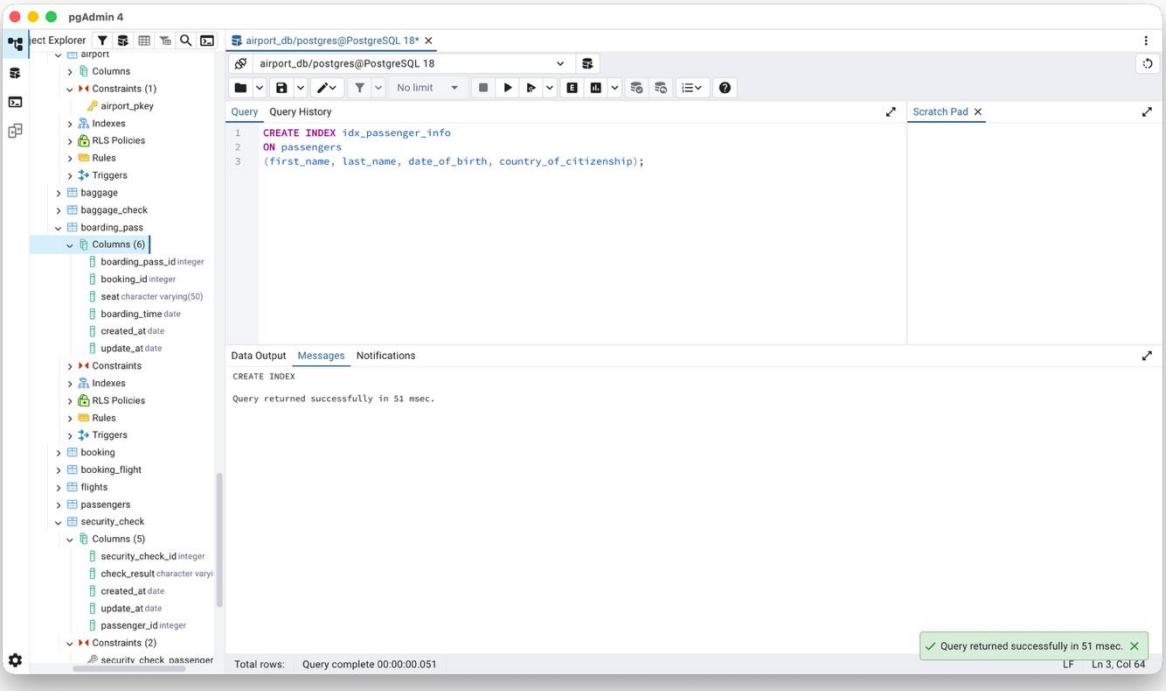
Step 4 - Check passenger passport values



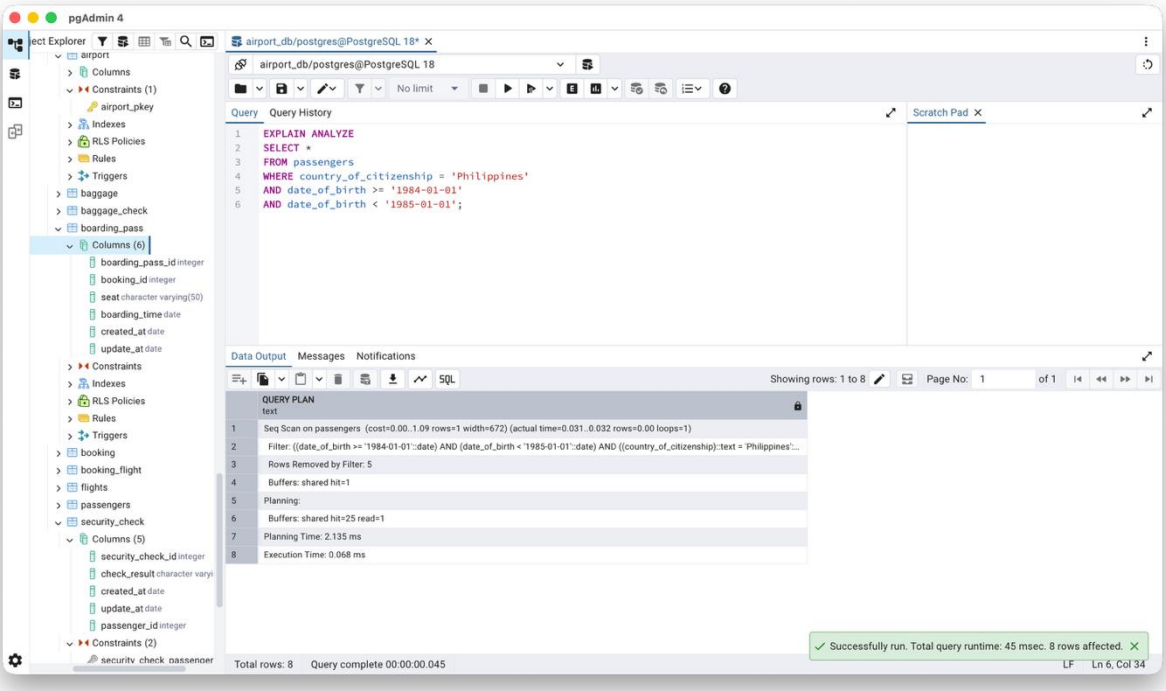
Explanation: The unique index was created for passport_number. It stops two passengers from using the same passport number. The screenshots do not show two successful new passenger inserts.

7. Create an index for the Passengers table. Use for that first name, last name, date of birth and country of citizenship. Then, write a SQL query to find a passenger who was born in Philippines and was born in 1984 and check if the query uses indexes or not. Give an explanation of the results.

Step 1 - Create a composite passenger index



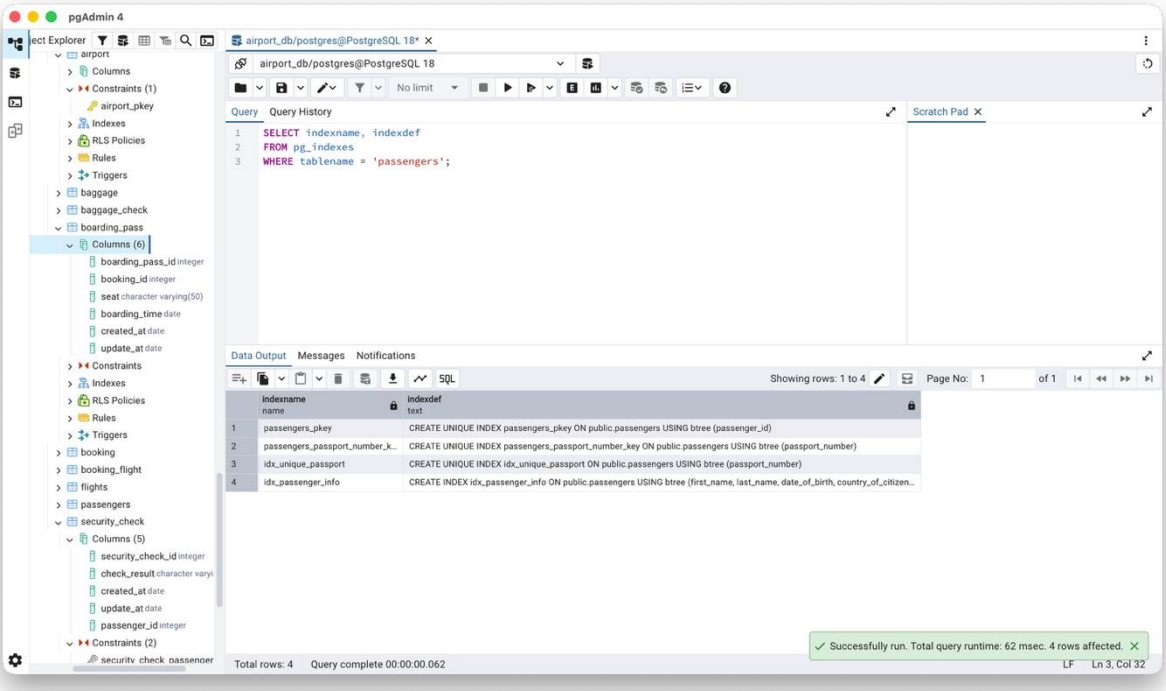
Step 2 - Check the query plan for passengers born in 1984



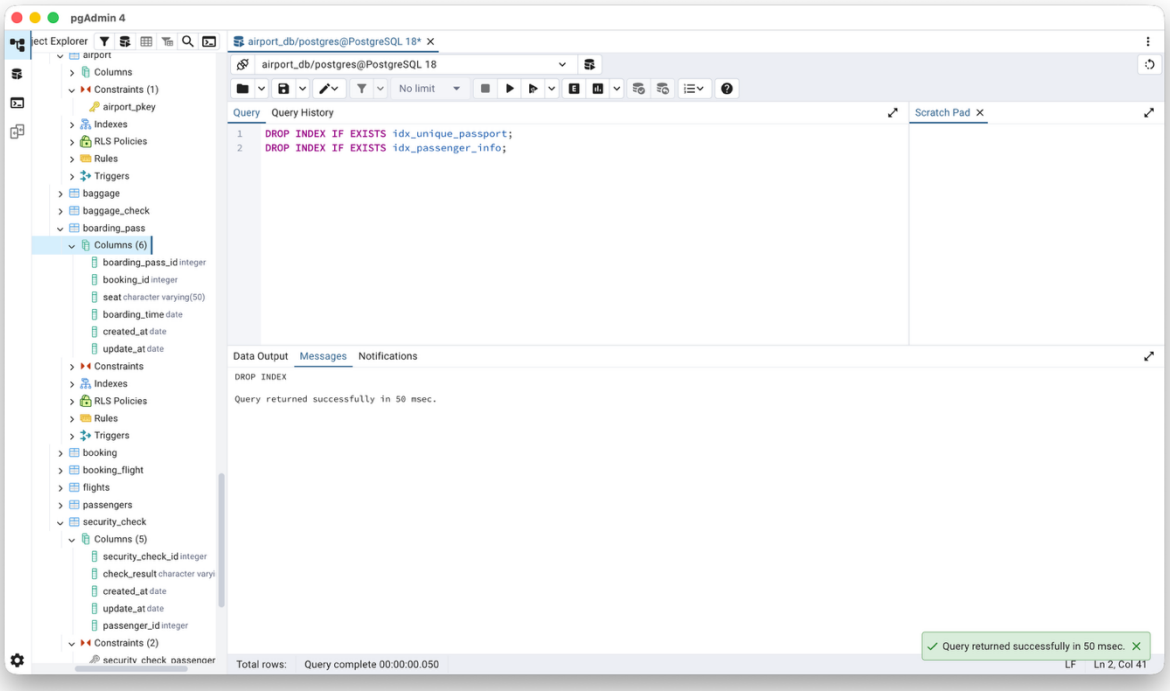
Explanation: I created an index using four passenger columns. The query searches by country and birth date. PostgreSQL used a Sequential Scan here; the first columns of the index are not in the filter.

8. Write a SQL query to list indexes for table Passengers. After, delete the created indexes.

Step 1 - Show existing indexes



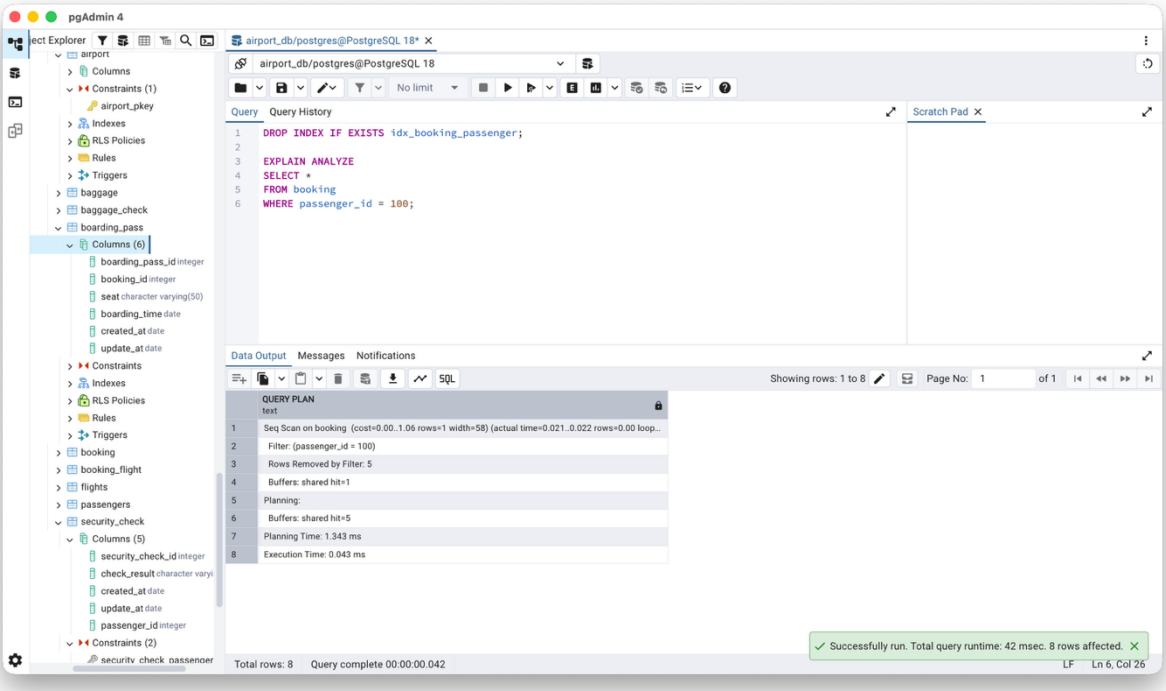
Step 2 - Delete the indexes created for passengers



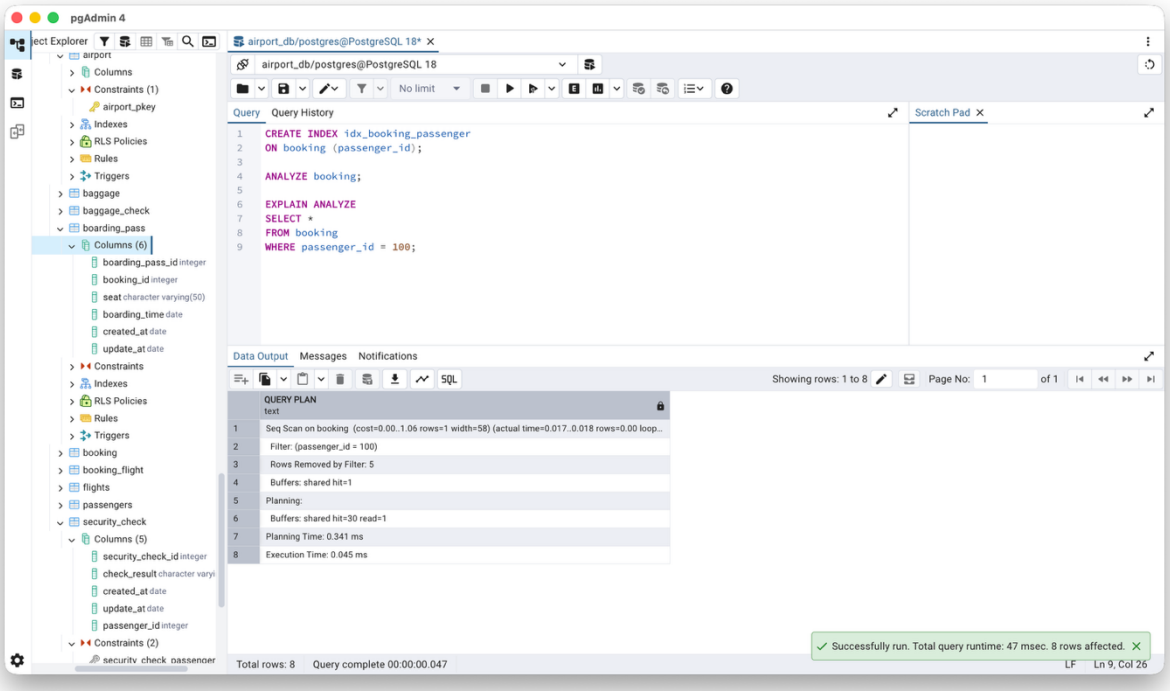
Explanation: I listed the indexes in passengers and removed the indexes created in this lab. The primary key index remains.

9. Create an index on the passenger_id column of the booking table. Using EXPLAIN ANALYZE, compare the execution plan and execution time of a query that selects all bookings of one passenger (for example, passenger_id = 100) before and after creating the index. Explain what changed in the plan.

Step 1 - Check the query before creating the index



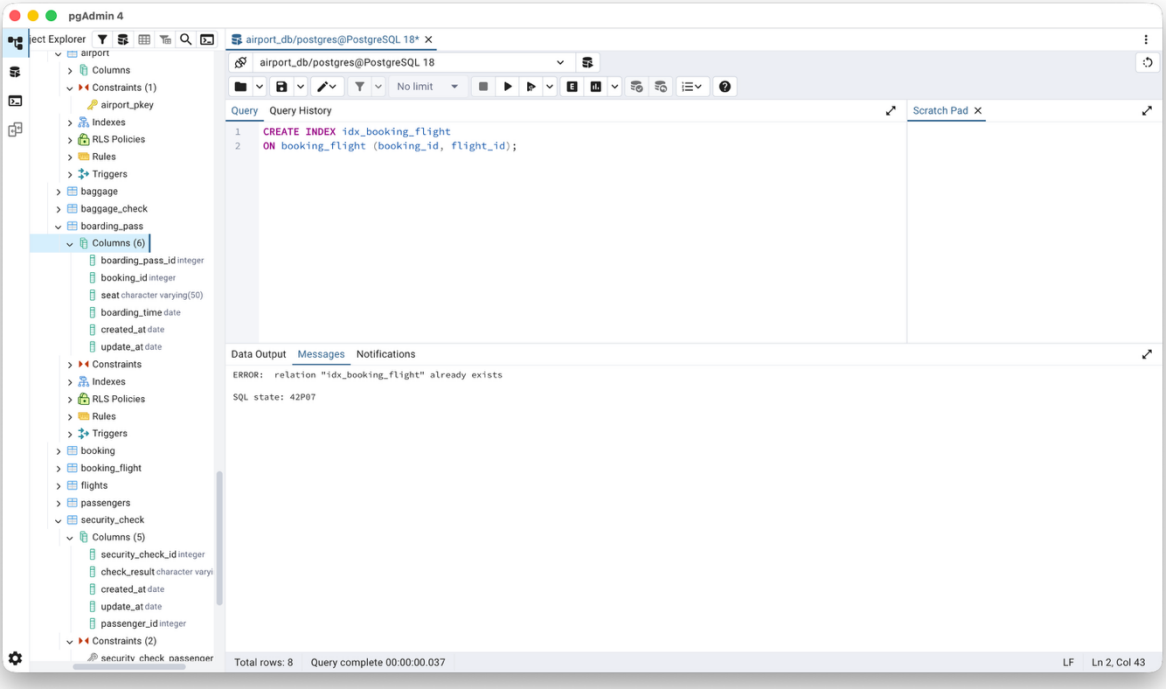
Step 2 - Create the index and test the query again



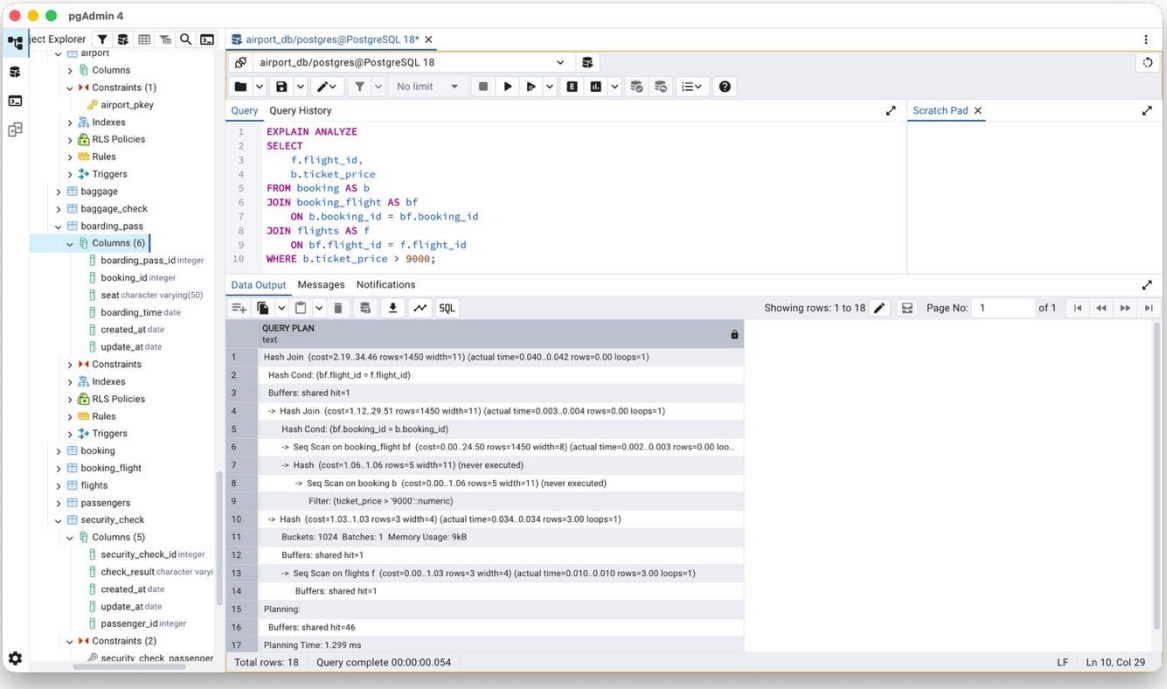
Explanation: I compared the query before and after adding the passenger_id index. The planner may choose a Sequential Scan for a small table, even when an index exists.

10. Create a composite index on the booking_flight table using the booking_id and flight_id columns. Write a query that joins booking, booking_flight and flights and returns the flight_no and price for bookings with a price greater than 9000. Check with EXPLAIN ANALYZE which indexes are used in each step of the join.

Step 1 - Create the booking-flight index



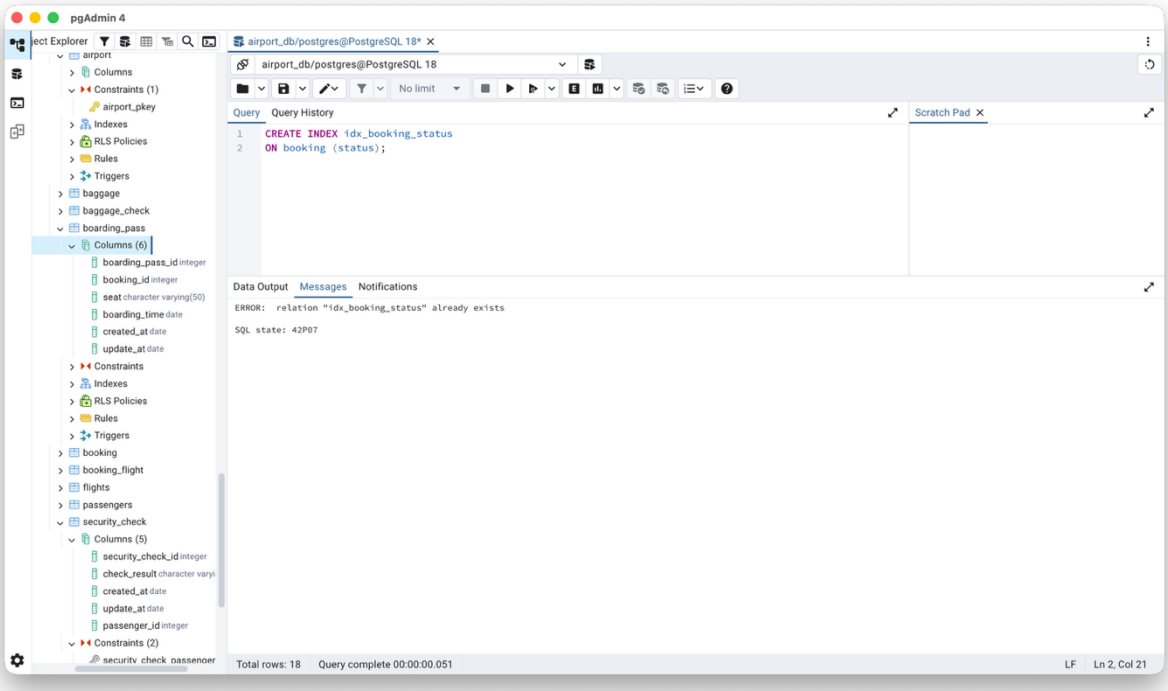
Step 2 - Check the join with EXPLAIN ANALYZE



Explanation: This query joins three tables and returns flight_id and ticket_price. It uses flight_id because flight_no does not exist in my database. EXPLAIN ANALYZE shows the steps of the join.

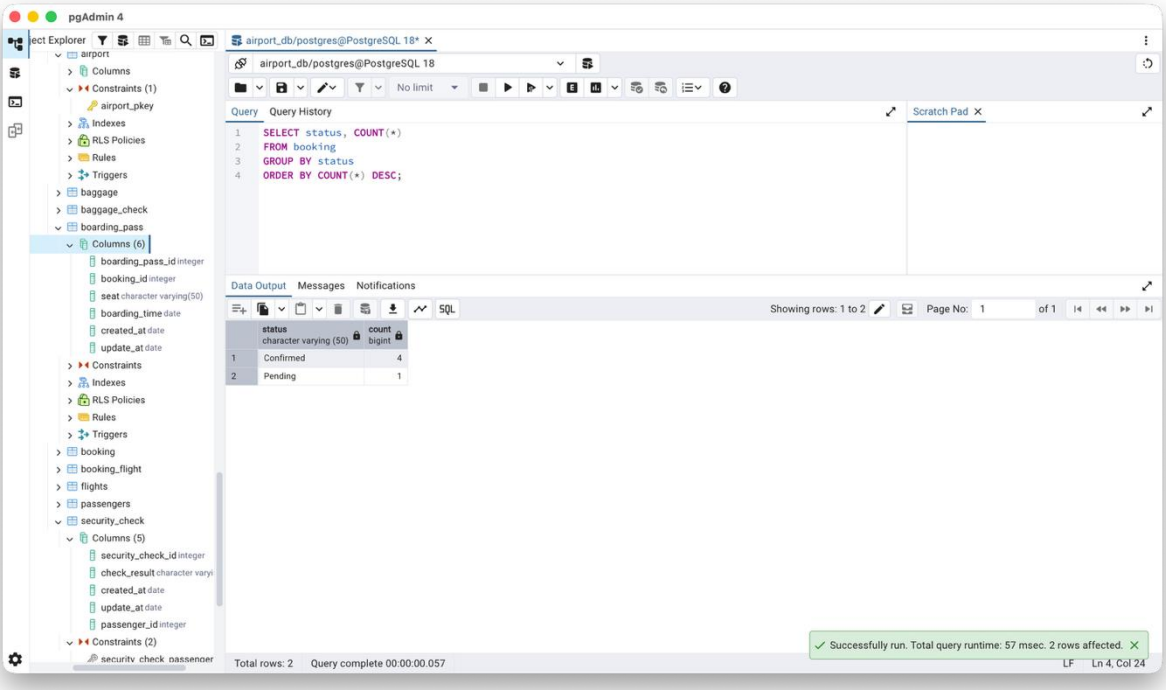
11. Create an index on the status column of the flights table. Run EXPLAIN ANALYZE for the query with status = 'Departed' and for the query with status = 'Boarding'. Compare the plans and explain, in your own words, why the optimizer may ignore the index for one of the values (hint: selectivity).

Step 1 - Check the status values and the execution plan

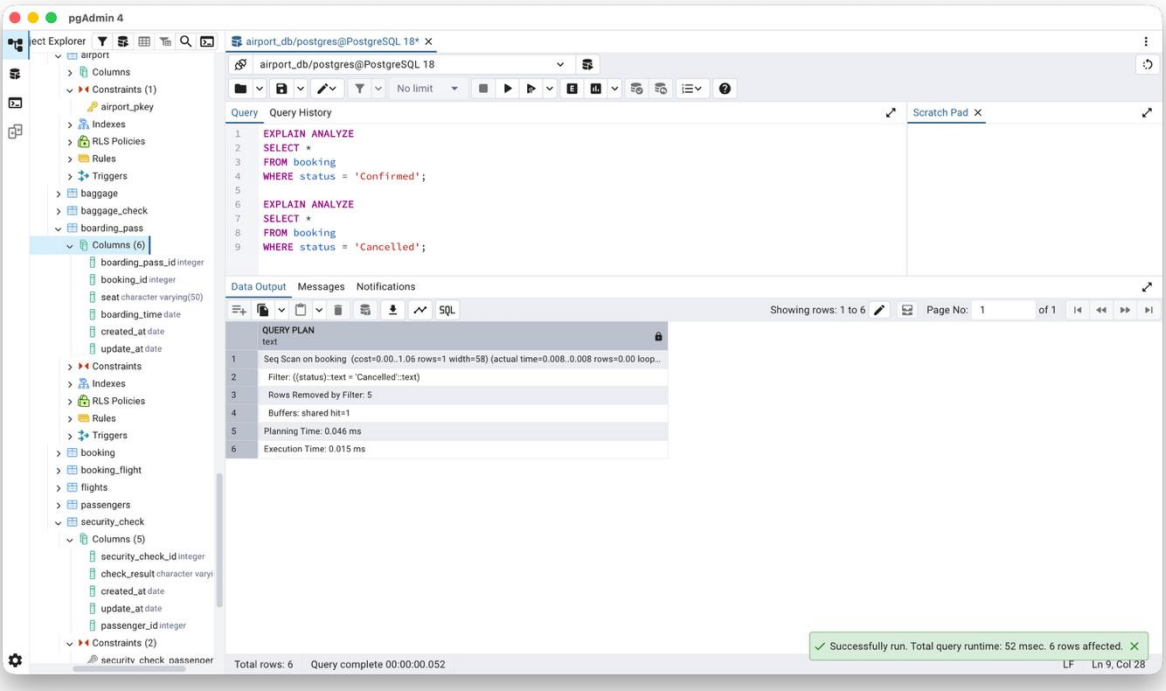


12.

Step 2 - Compare the status query plans

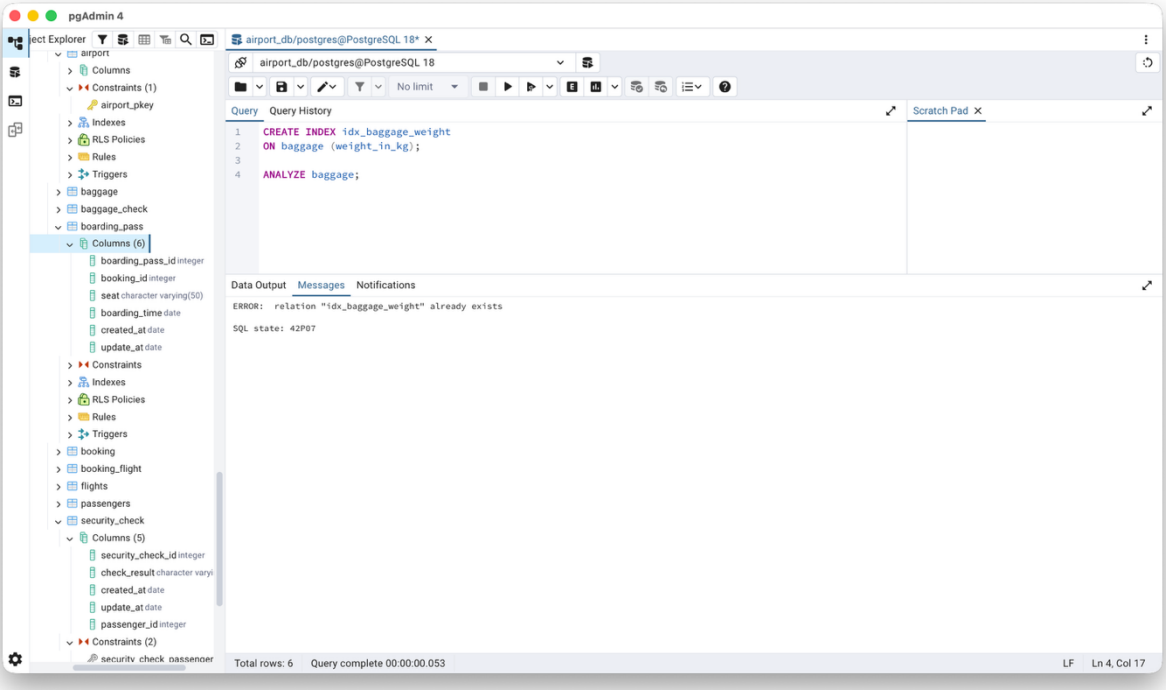


Step 3 - Review the query result

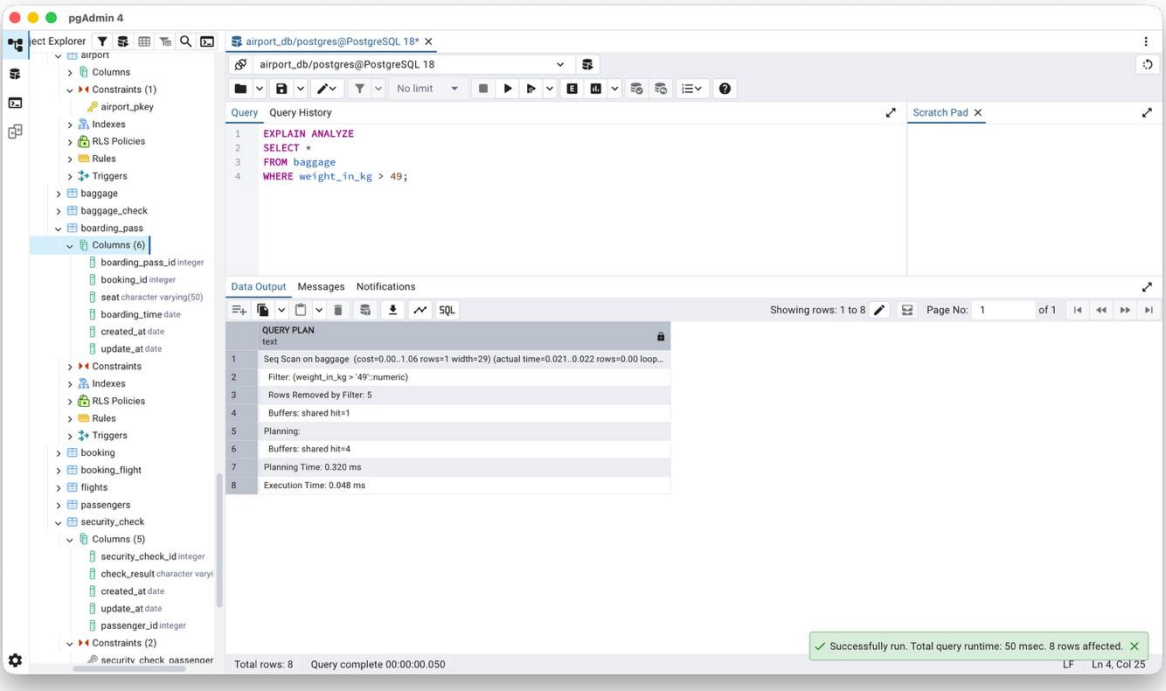


12 Create an index on the weight_in_kg column of the baggage table. Compare the plans of two range queries: weight_in_kg > 49 (a small part of the table) and weight_in_kg > 5 (almost the whole table). Explain the difference in index usage.

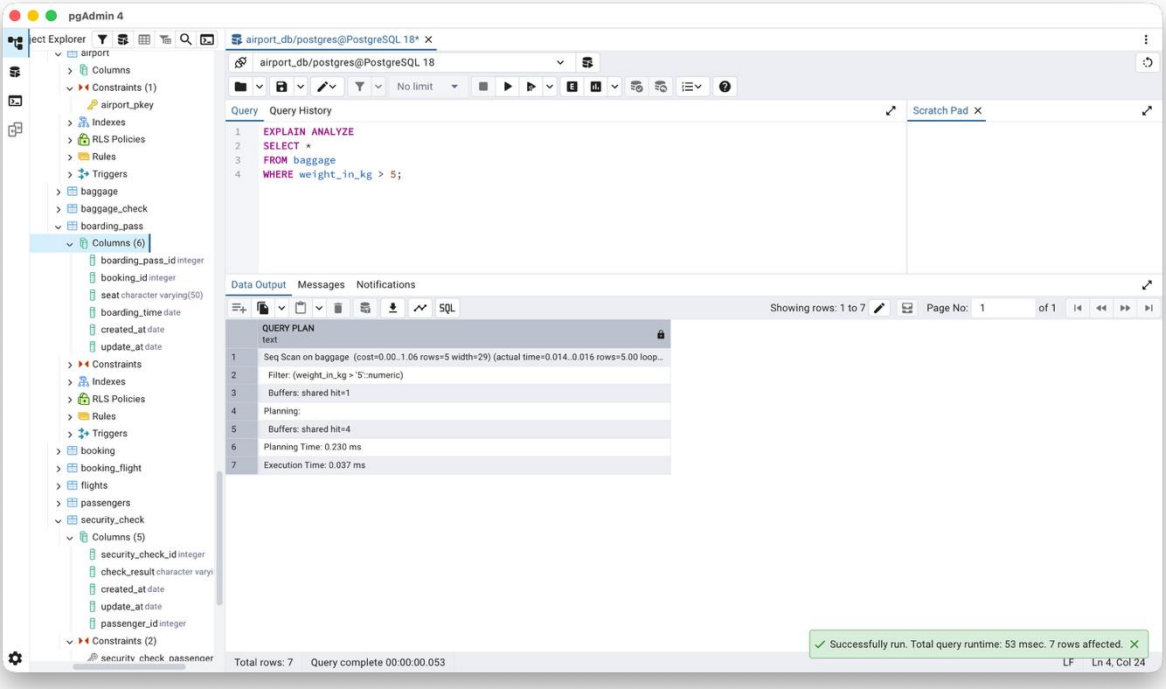
Step 1 - Create the baggage weight index



Step 2 - Check the query for weights above 49 kg



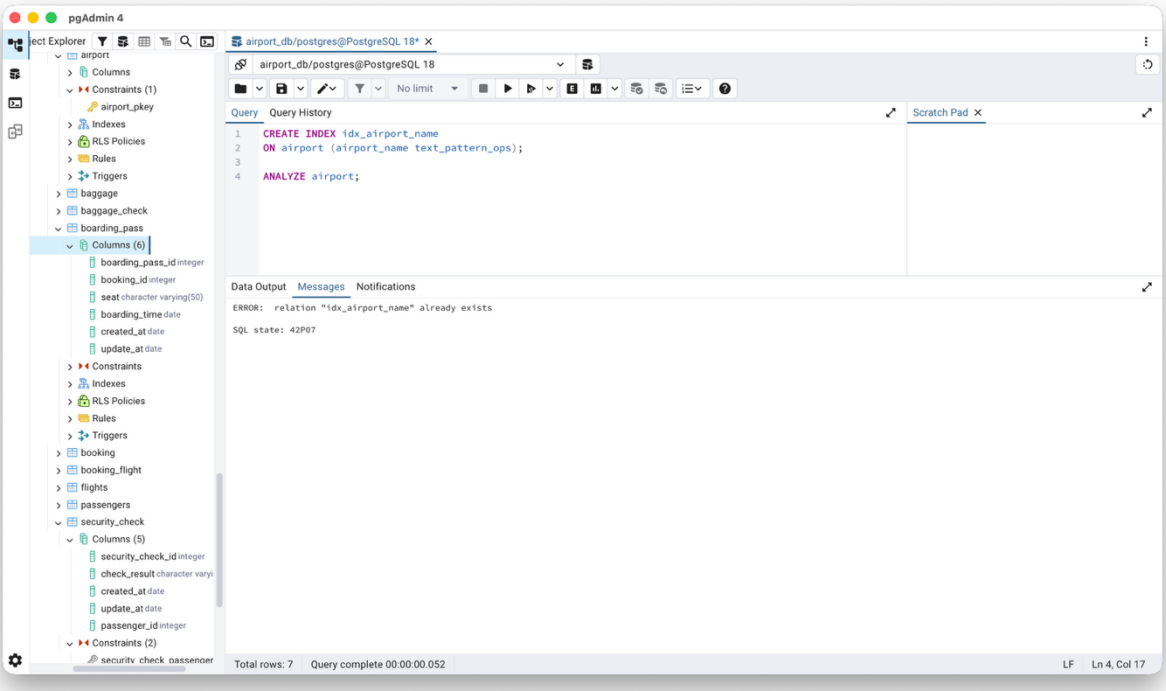
Step 3 - Check the query for weights above 5 kg



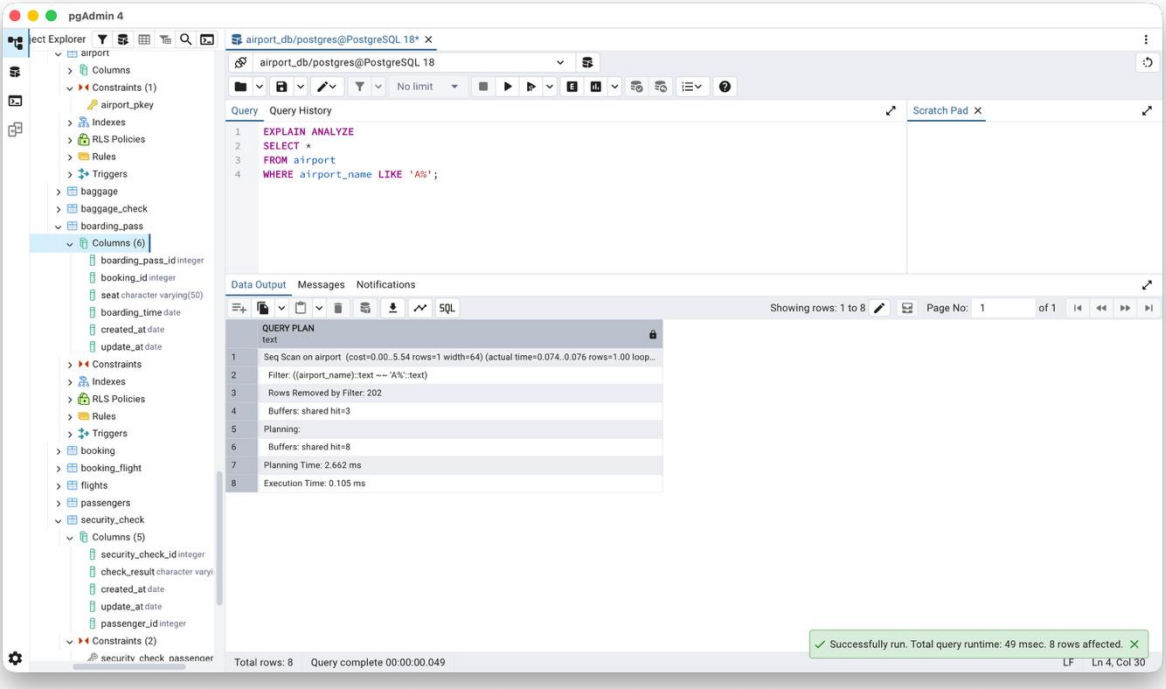
Explanation: This is an alternative example using booking.status because flights.status is not in my database. A common value can make a full table scan cheaper than an index scan. The planner chooses the lower-cost plan.

13. Create an index on the airport_name column of the airport table. Check with EXPLAIN ANALYZE whether it is used by the queries LIKE 'A%' and LIKE '%Airport'. Explain why the results are different.

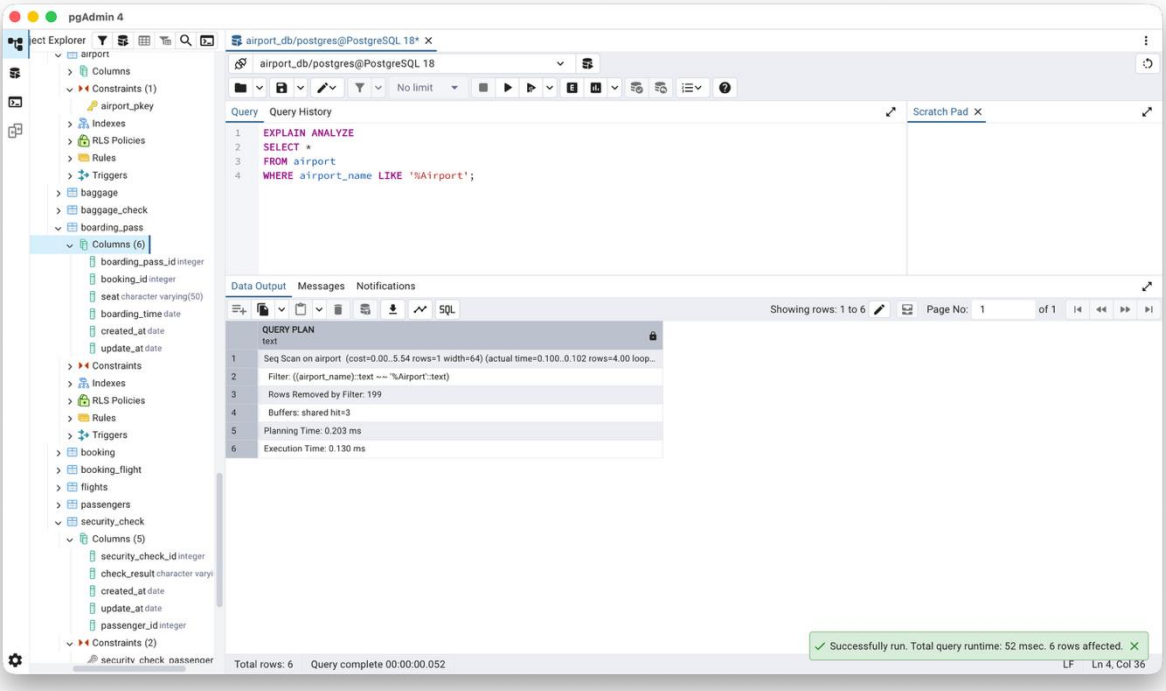
Step 1 - Test airport names starting with A



Step 2 - Test airport names ending with Airport



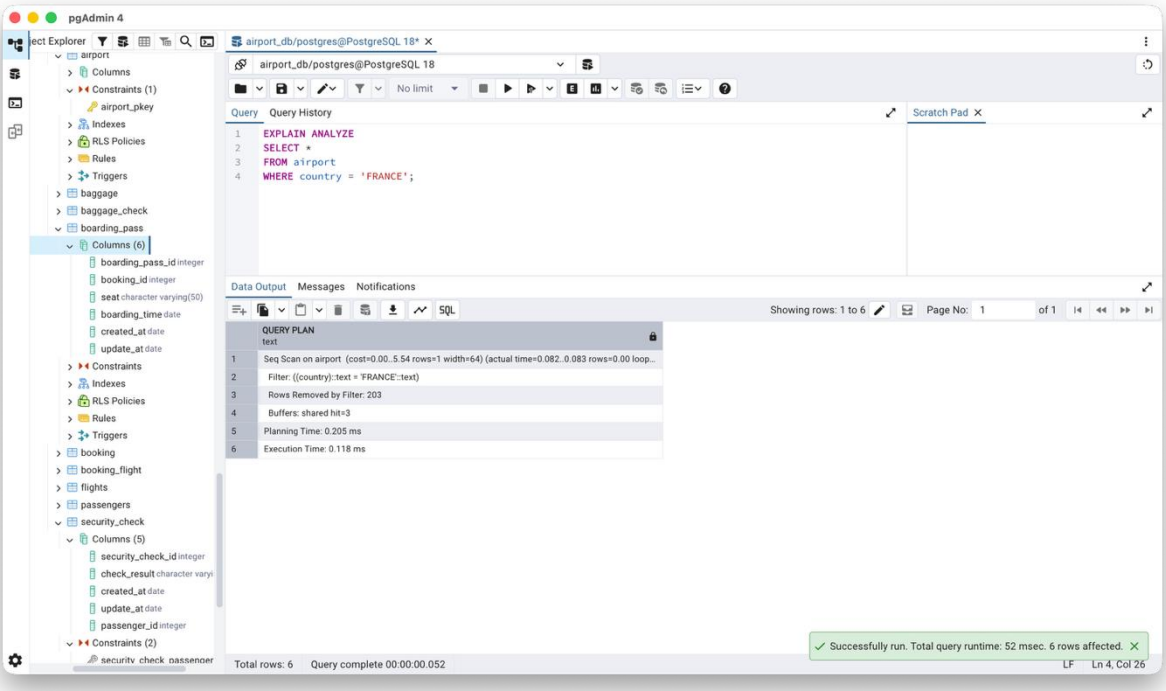
Step 3 - Create or check the airport name index



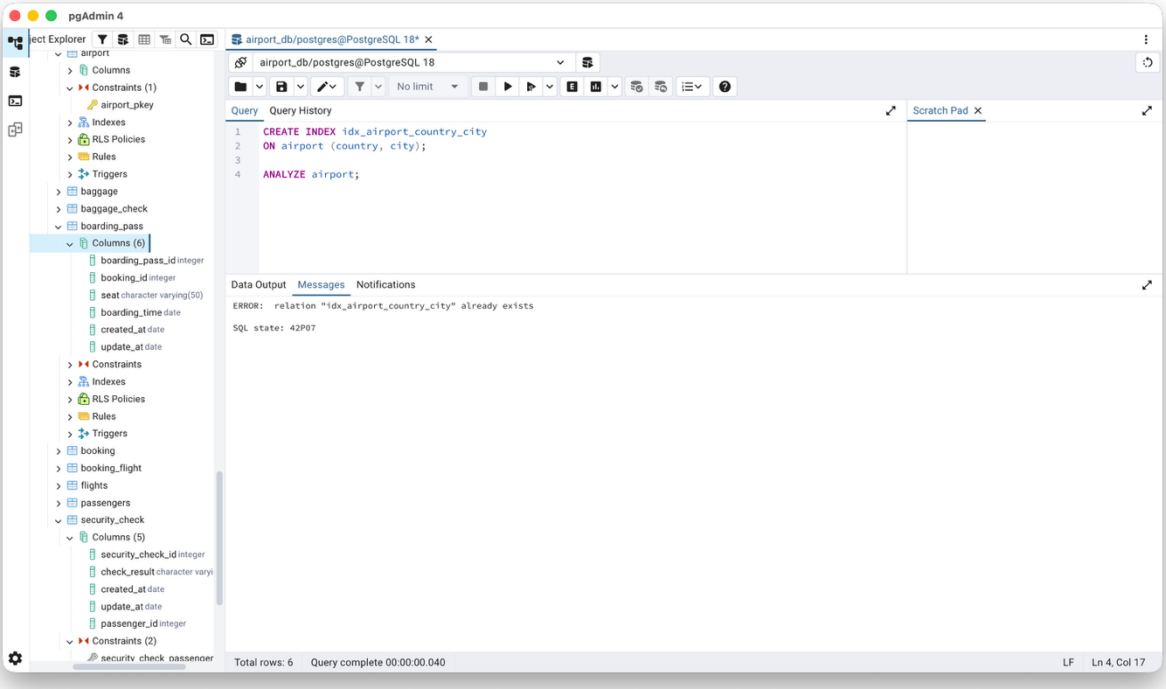
Explanation: The first query finds a small range of weights. The second query may return most rows. PostgreSQL may use the index for a small result and a Sequential Scan for a large result.

Explanation: LIKE 'A%' searches from the start of the name, so a suitable B-tree index can help. LIKE '%Airport' starts with a wildcard, so the same index is usually not useful.

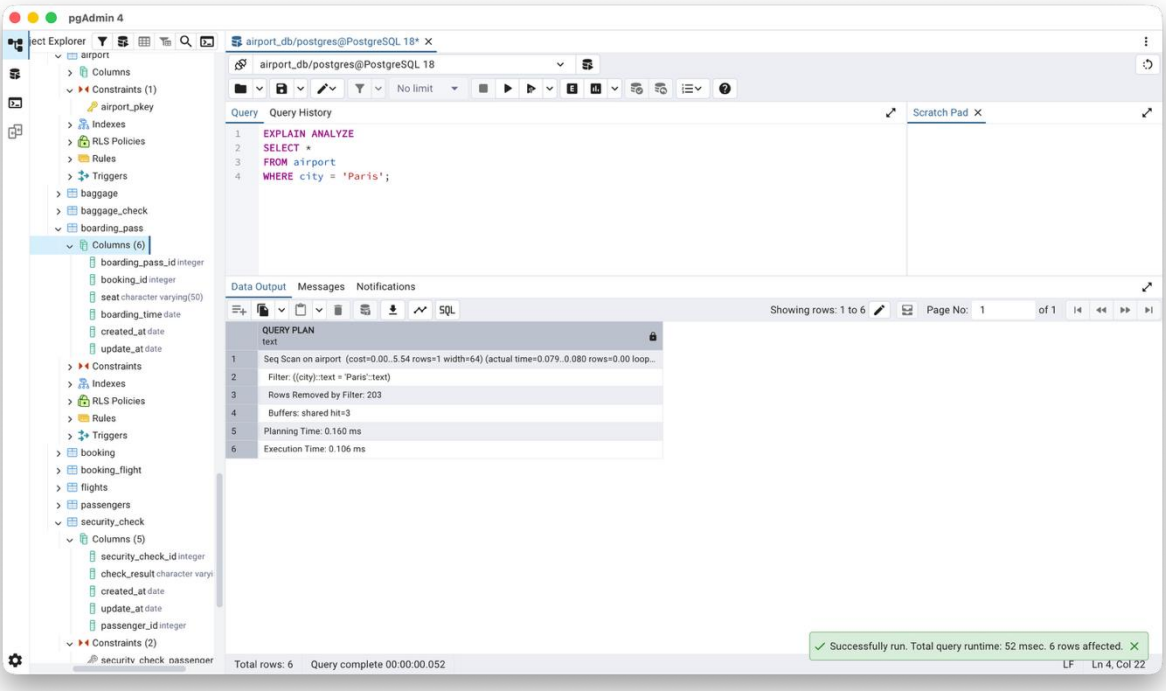
14. Create a composite index on the country and city columns of the airport table. Run three queries: filtering by country only, by city only, and by both columns. Check which of them use the index and explain the leftmost prefix rule using your results.
Step 1 - Compare the airport query plans



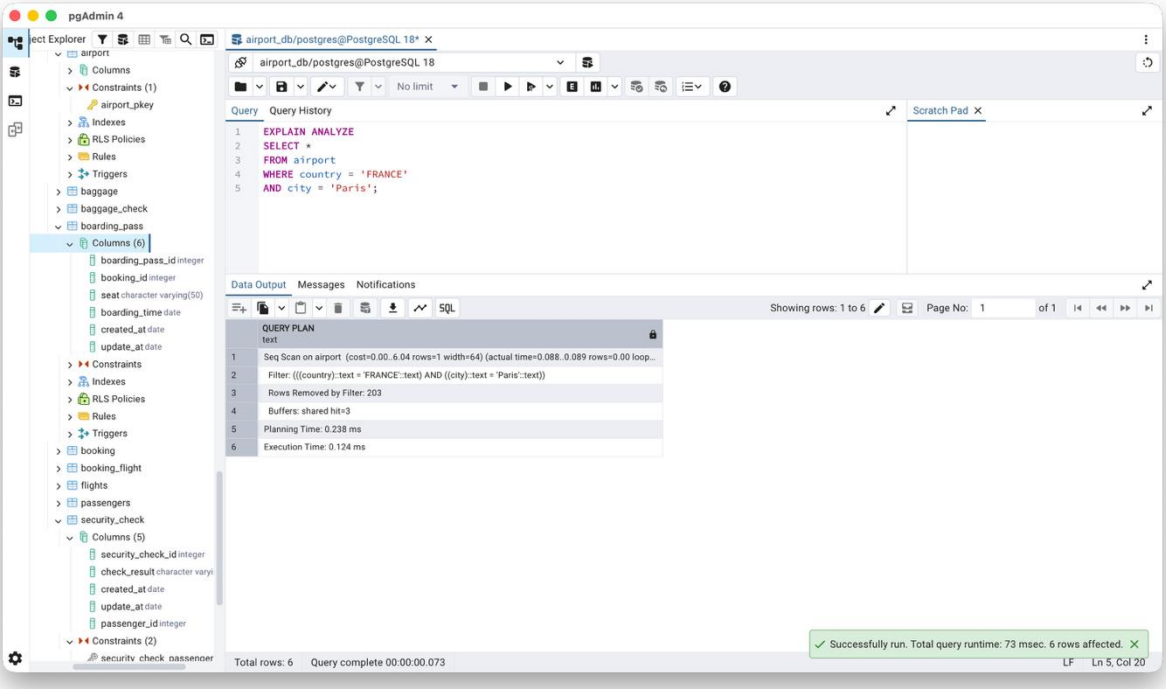
Step 2 - Check a country filter



Step 3 - Create the country and city index



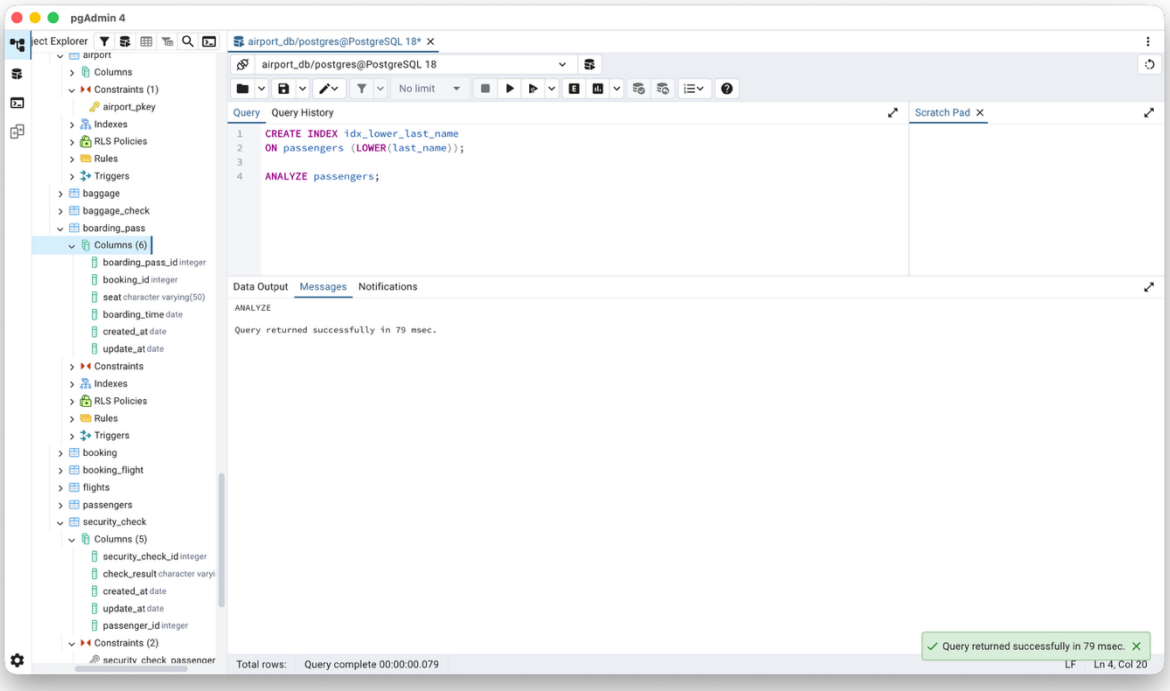
Step 4 - Check the city filter



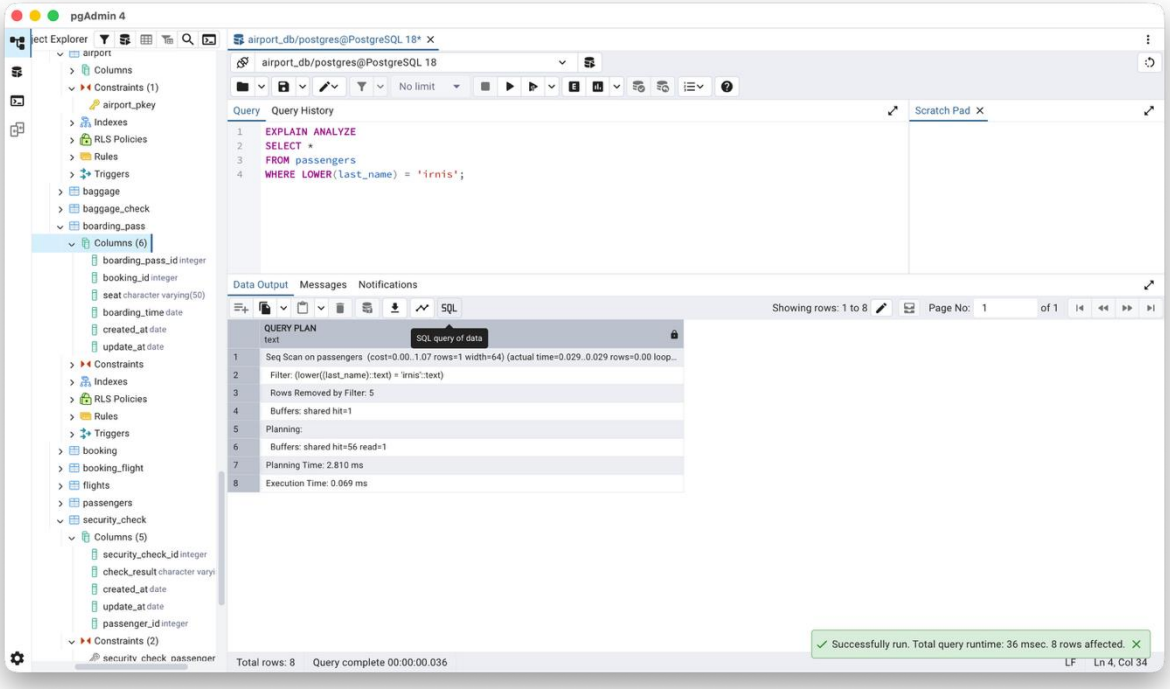
Explanation: The index starts with country and then city. A search by country or by both columns can use the leftmost part of the index. A search by city only is usually less useful for this index.

15. Create an expression (functional) index on the lower-case last_name of the Passengers table (LOWER(last_name)). Compare the plans of a query written as WHERE LOWER(last_name) = 'irnis' and a query written as WHERE last_name = 'Irnis'. Explain which query can use the index and why.

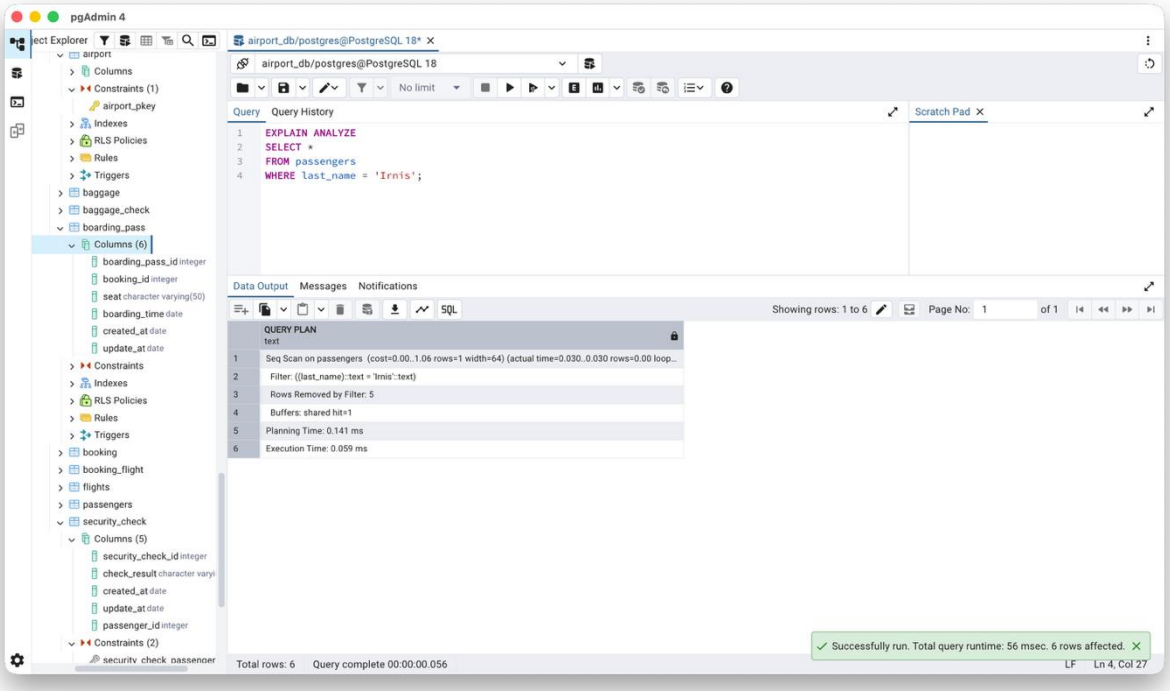
Step 1 - Compare both last-name queries



Step 2 - Create the LOWER(last_name) index



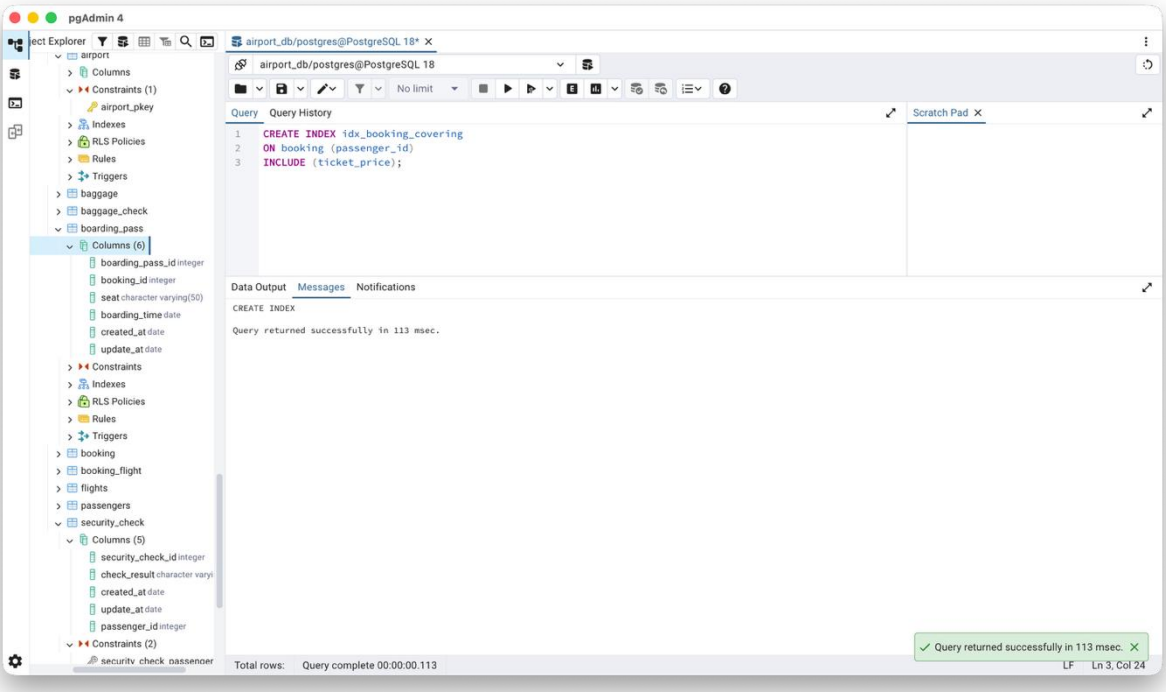
Step 3 - Review the execution plan



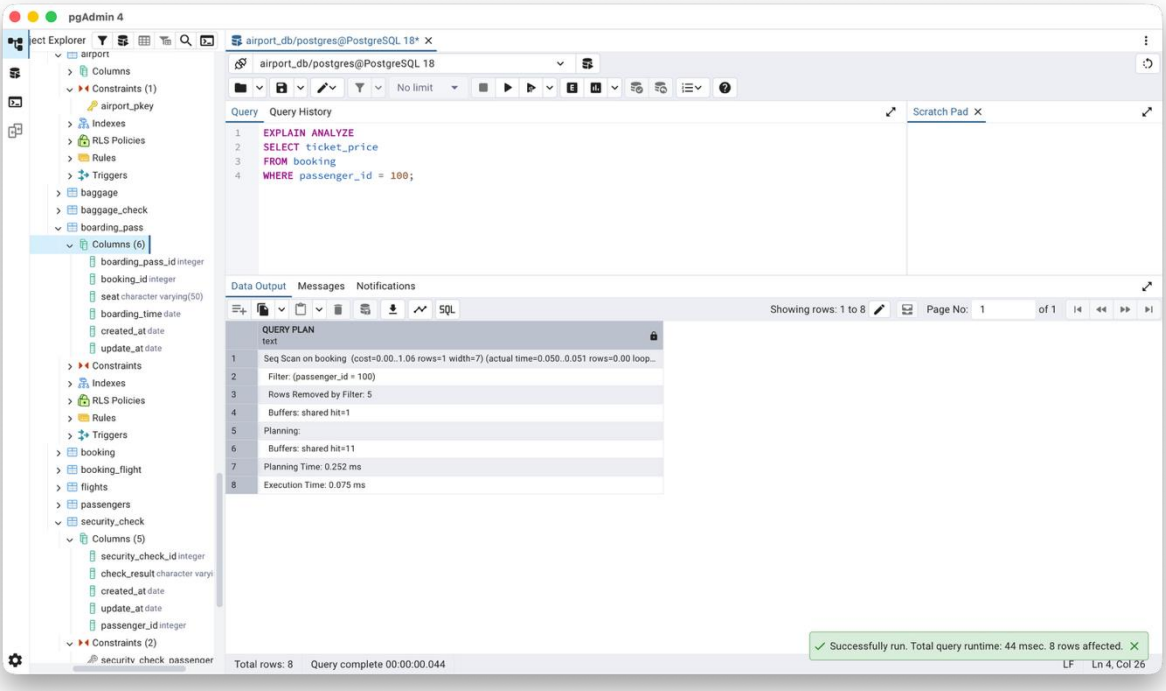
Explanation: The expression index stores LOWER(last_name). The query with LOWER(last_name) can use it. The query with last_name = 'Irnis' does not match that expression index.

16. Create a covering index on the booking table for the column passenger_id and price. Write a query that returns only price for a given passenger_id and check with EXPLAIN ANALYZE whether an index-only scan (or "Using index") is used. Explain the benefits of a covering index.

Step 1 - Check the query with passenger_id



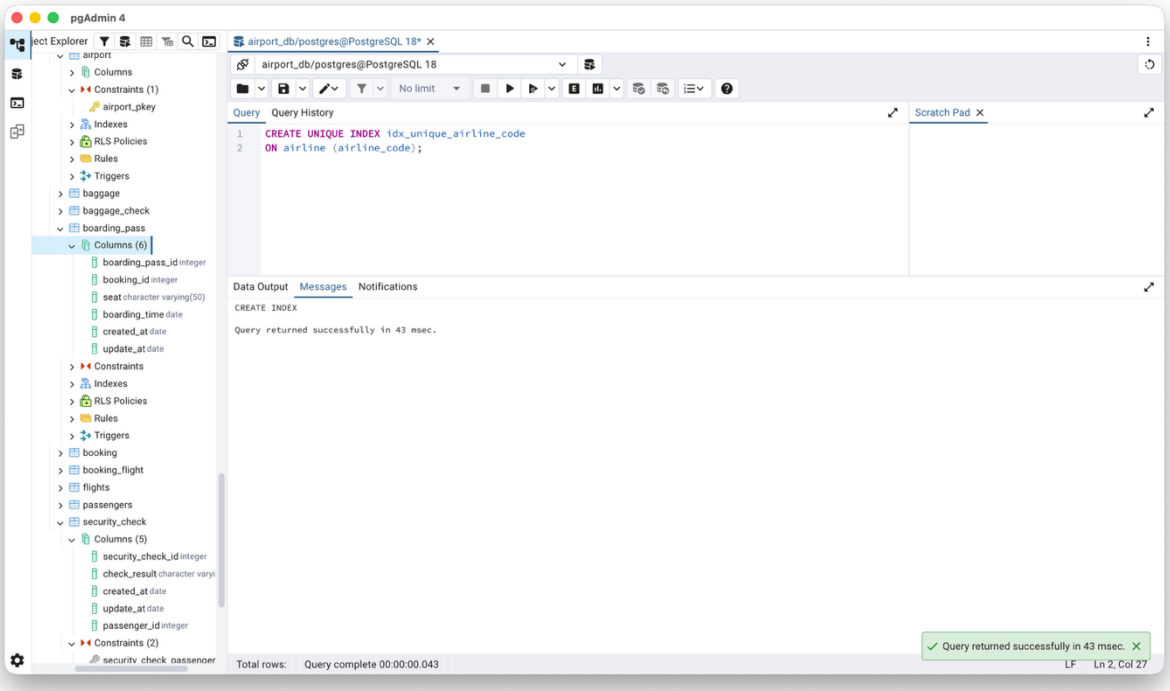
Step 2 - Check the query with a covering index



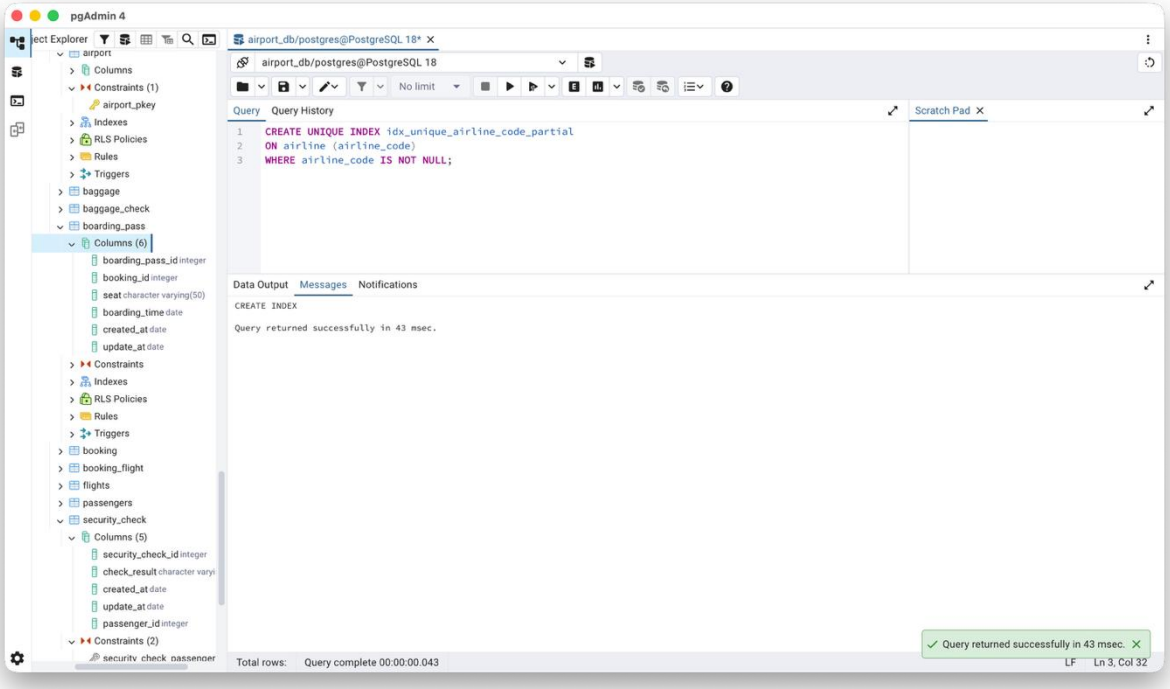
Explanation: The covering index stores passenger_id and ticket_price. It can help PostgreSQL get the price from the index only. In this result PostgreSQL used a Sequential Scan, so an Index Only Scan was not chosen.

17. Try to create a unique index on the airline_code column of the airline table. If an error occurs, explain what caused it and find the problematic rows with a GROUP BY ... HAVING COUNT(*) > 1 query. Then create the unique index in a way that works for the current data, or after fixing the data.

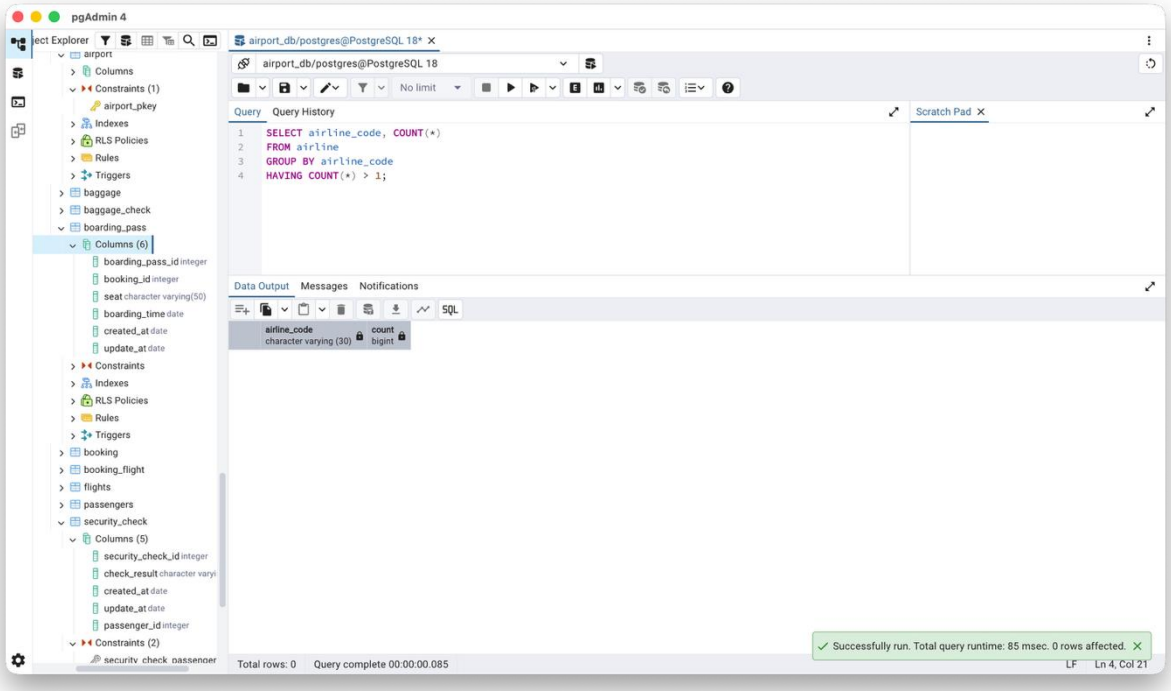
Step 1 - Try creating a unique airline code index



Step 2 - Check the existing data



Step 3 - Find duplicate airline codes



Explanation: The duplicate check returned no duplicate airline codes. The unique index was created successfully. It prevents the same non-null airline code from being stored twice.